AgroChat: Agriculture Question Answering with Sentence Embeddings

GitHub Repository: https://github.com/f-Ayuk/AgroChat Demo Video: https://www.loom.com/share/0ede7f7165ed4d0c8f38b57b84e2a208. Live System: https://agrochat-blifo2jrrhz3yxzba3jtep.streamlit.app/#agro-chat

1. Abstract

AgroChat is an English-language agriculture question-answering prototype. It retrieves an answer from a fixed question-and-answer collection rather than generating new advice. This project fine-tunes the `all-MiniLM-L6-v2` sentence encoder on agriculture QA pairs and compares it with the original pretrained model. On the test split, the fine-tuned model achieved 28.02% top-1 exact answer accuracy, compared with 22.20% for the baseline. These are exploratory results: duplicate questions cross the data splits, and the out-of-domain test contains only 12 questions. The system is English-only and does not address African-language NLP.

2. Introduction

Farmers and other users may need quick information about soil, crops, irrigation, pests, and harvesting. AgroChat tests a simple retrieval approach: find the stored question most similar to the user's question, then return its associated answer. If the similarity is below a threshold, the system returns a fallback message.

The project asks whether agriculture-specific fine-tuning improves retrieval over a general-purpose sentence encoder, and how the similarity threshold affects in-domain acceptance and out-of-domain rejection. The intended user is an English-speaking person seeking general farming information. The current system does not support an African language.

3. Related Work

Sentence-BERT produces sentence embeddings that can be compared efficiently for semantic search (Reimers & Gurevych, 2019). AgroChat uses the `all-MiniLM-L6-v2` sentence-transformer, a compact model based on MiniLM (Wang et al., 2020).

The model is fine-tuned with `MultipleNegativesRankingLoss`. For each question-answer pair, the paired answer is positive and other answers in the batch act as negatives (Sentence Transformers, n.d.). This is a common retrieval-training approach, though repeated answers in this dataset may become false negatives.

4. Dataset and Data Preparation

The preparation script downloads the `train` split of `talhakk/agriculture-qa`. Its dataset card reports 25,410 English question-answer pairs and an Apache-2.0 license (talhakk, n.d.). The project has not independently verified how the answers were collected, who wrote them, or whether they were reviewed by agricultural experts.

Empty questions and answers are removed. The remaining data is split sequentially at random into 20,328 training, 2,541 validation, and 2,541 test pairs using seed 42. The split is not stratified or deduplicated. The generated `category` and `crop_tag` fields are blank.

Duplicates cross the splits: 419 test questions also appear in training, and 325 exact question-answer pairs overlap between training and test. This weakens the independence of the evaluation and may inflate results. The English-only dataset also lacks verified local vocabulary and region-specific guidance. Future work should document data provenance, deduplicate before splitting, and collect reviewed questions from intended users.

5. Methodology

Baseline and model

The baseline is `sentence-transformers/all-MiniLM-L6-v2` without agriculture-specific fine-tuning. The fine-tuned model starts from the same encoder. Its Transformer has six layers and produces 384-dimensional sentence vectors. Mean pooling combines token representations; L2 normalization prepares the vectors for cosine similarity.

Training

`scripts/train_embedder.py` trains on question-answer pairs using `MultipleNegativesRankingLoss`, batch size 64, and three epochs. The data loader shuffles the training pairs. Training uses 10% warm-up steps, and the validation evaluator compares validation questions with validation answers.

The script does not explicitly set a learning rate or random seed, and `requirements.txt` does not pin the SentenceTransformers version. In the environment used for this evaluation, the library's default learning rate was 2e-5. Reproduction should set the learning rate and seeds explicitly and pin dependencies. Repeated answers may also act as false negatives during training.

Retrieval

At startup, the app embeds the questions in `data/qa_train.csv`. For a new question, it computes an embedding and compares it with the stored vectors. Since the vectors are normalized, their dot product equals cosine similarity. The app returns the answer paired with the closest training question if the score is at least 0.45; otherwise, it returns a fallback. The score is a similarity measure, not a probability that the answer is correct. The app retrieves stored text and does not generate or verify advice.

6. Experiments and Results

The test questions are searched against the training-question index, matching the application's retrieval process. Top-1 accuracy is an exact answer comparison: different wording is counted as incorrect even if the meaning is similar. In-domain acceptance is the share of test questions at or above the threshold. OOD rejection is the share of 12 manually written non-agriculture questions below it.

Table 1. Results at the app's default threshold of 0.45

Model	Top-1 exact answer	In-domain accepted	OOD rejected
Pretrained MiniLM	22.20%	99.88%	10/12 (83.33%)
Fine-tuned MiniLM	28.02%	99.88%	10/12 (83.33%)

Fine-tuning improved exact-answer retrieval by 5.82 percentage points. The threshold changes the rejection trade-off: at 0.55, both models reject all 12 OOD questions, while in-domain acceptance falls to 99.21% for the baseline and 98.58% for the fine-tuned model.

The reported pooled F1 is approximately 0.999, but it is dominated by the imbalance between 2,541 in-domain examples and 12 OOD examples. It should not be read as evidence of robust OOD detection. The small OOD sample is also coarse: one question changes rejection by 8.3 percentage points. Results are further limited by duplicate questions across splits. A larger, balanced, independently reviewed test set is needed.

7. Error Analysis and Discussion

Across the 2,541 test questions, fine-tuning produced 155 exact-answer wins over the baseline; the baseline produced seven wins over fine-tuning. Both models matched 557 answers exactly, and neither matched exactly for 1,822 questions.

For "asking about control of termite in sugarcane," fine-tuning retrieved a related South Asia entry and returned the exact expected answer. The baseline retrieved a near-duplicate question but a different answer. Its similarity score was higher (0.9965 versus 0.9757), showing that a higher score does not necessarily mean a better answer. The source answer contains treatment advice and is not reproduced here as a recommendation.

For a Dinanath grass query, the baseline returned the expected answer while fine-tuning selected a different answer about production practices. Exact matching cannot determine whether those answers are semantically equivalent. For a sali paddy sowing-time query, both systems retrieved a related entry. The test answer says "May-June," while the retrieved answer says "mid May to mid June"; exact matching marks both wrong despite the similar time period.

These examples show the limits of exact-match scoring and the difficulty of mapping similar questions to the right stored answer. Human review and independently collected farmer questions would provide a stronger evaluation.

8. Deployment

The Streamlit interface calls the shared `retrieve_answer` function in `app.py`. It loads the knowledge-base CSV and fine-tuned model from the GitHub repository; no separate model service is used. A Flask interface and API are also included.

Live app: No public Streamlit URL has been recorded. Add and test the actual URL after deployment before including it in the final submission.

9. Limitations and Future Work

- **Language:** The app and dataset are English-only. They do not meet an African-language NLP objective.
- **Data:** Source provenance and answer quality have not been independently verified; duplicate pairs cross the splits.
- **Evaluation:** The OOD set has only 12 questions, and exact matching penalizes answers with different wording.
- **Reproducibility:** Pin dependencies and set training seeds and learning rate explicitly.
- **Safety and locality:** Advice may be outdated or unsuitable for local conditions. Users should consult qualified local agricultural advisers.
- **Unintegrated code:** `scripts/train_classifier.py` expects classifier CSV files that are not included. No trained classifier is used by the app.
- **Deployment:** A working public Streamlit URL still needs to be recorded.

Next steps are to improve data provenance, remove duplicate leakage, build a larger reviewed evaluation set, and—if required by the assignment—develop and evaluate the system for a selected African language.

10. Conclusion

AgroChat demonstrates a practical sentence-embedding retrieval workflow with fine-tuning, a pretrained baseline, threshold tests, and a web interface. Fine-tuning improved exact-answer retrieval on the current test split, but duplicate leakage, a small OOD set, and English-only data limit the conclusions. The system is a prototype, not a verified agricultural advice service or an African-language NLP solution.

11. References

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proceedings of EMNLP-IJCNLP*, 3973–3983. https://doi.org/10.18653/v1/D19-1410

Sentence Transformers. (n.d.). *MultipleNegativesRankingLoss documentation*. https://www.sbert.net/docs/package_reference/sentence_transformer/losses.html

Sentence Transformers. (n.d.). *all-MiniLM-L6-v2 model card*. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

talhakk. (n.d.). *Agriculture QA* [Data set]. Hugging Face. https://huggingface.co/datasets/talhakk/agriculture-qa

Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., & Zhou, M. (2020). MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *Advances in Neural Information Processing Systems, 33*.